Machine Learning Term Project Team 4

Final Report

Predicting Yelp Review Star Ratings:
A Comparative Study of Traditional and Deep Learning Methods

Alexander Nardi, Anthony Mahon, Benedetto Falin, Keating Sane, Mohammed Mamdouh

May 4, 2026

Contents

1 Team Information **3**
1.1 Team Members and Roles . . . 3
1.2 Responsibility Distribution . . . 3
1.3 Team Assignment Table . . . 3

2 Problem Setup **4**
2.1 Classification Task . . . 4
2.2 Dataset . . . 4
2.3 Dataset Size and Splits . . . 4
2.4 Input and Output Format . . . 4
2.5 Class Labels . . . 4
2.6 Basic Dataset Statistics . . . 5
2.7 Project Goals . . . 5

3 Implementation Description **5**
3.1 Repository and Codebase Overview . . . 5
3.2 Data Preparation and Experimental Pipeline . . . 6
3.3 Traditional Machine Learning Models . . . 8
3.4 Deep Learning Models . . . 12
3.5 Cross-Model Engineering Choices . . . 15

4 Usage Guide **16**
4.1 Environment Setup . . . 16
4.2 Common Run Modes . . . 16
4.3 How to Run Each Model . . . 17
4.4 Expected Output Files . . . 18

5 Evaluation Metrics **18**
5.1 Notation . . . 18
5.2 Accuracy . . . 18
5.3 Macro Precision . . . 19
5.4 Macro Recall . . . 19
5.5 Macro F1-Score . . . 19
5.6 Confusion Matrix . . . 19
5.7 Why These Metrics Fit This Project . . . 19

6 Results **19**
6.1 Validation Experiments . . . 19
6.2 Final Experiments . . . 19
6.3 Hardware and Software Environment . . . 19
6.4 Tuning Budget . . . 19
6.5 Experiment Tracking . . . 19
6.6 Comparison Table . . . 19
6.7 Per-Model Result Summaries . . . 21
6.8 Visualizations . . . 21
6.9 Comparative Discussion . . . 21

7 Conclusions & Future Work **21**
7.1 Key Findings . . . 21
7.2 Lessons Learned . . . 21
7.3 Limitations . . . 21
7.4 Future Improvements . . . 21
7.5 Final Takeaway . . . 21

1. Team Information

1.1 Team Members and Roles

1.2 Responsibility Distribution

1.3 Team Assignment Table

Team Member	Models
Alexander Nardi (Project Coordinator)	Linear SVM, LSTM
Anthony Mahon	Kernel SVM, Transformer encoder
Benedetto Falin	Naive Bayes, Multi Layer Perceptron
Keating Sane	Decision Tree/Random Forest, CNN
Mohammed Mamdouh	Logistic Regression, LLM

2. Problem Setup

2.1 Classification Task

The task is five-class single-label text classification. Given the raw text of a Yelp review, the model must predict the star rating the reviewer assigned, which is one of five ordinal classes ranging from 1 star to 5 stars. Each review belongs to exactly one class. The ordinal structure of the labels means that misclassifying a 1-star review as 2-star is a smaller error than misclassifying it as 5-star, though all standard metrics used here treat all misclassifications equally.

2.2 Dataset

The dataset used is **Yelp Review Full**, a large-scale sentiment classification benchmark compiled from the 2015 Yelp Dataset Challenge. It consists of English-language Yelp business reviews paired with their star ratings. Reviews vary considerably in length, from a single sentence to several paragraphs, and cover a wide range of business categories. The dataset is publicly available through HuggingFace Datasets under the identifier `yelp_review_full`.

2.3 Dataset Size and Splits

Split	Samples	Notes
Training	585,000	90% of the official 650,000-review training partition
Validation	65,000	10% stratified split carved from the training partition
Testing	50,000	Official held-out test partition, never seen during tuning

2.4 Input and Output Format

Input: Raw English review text as a variable-length string. No manual preprocessing is required before passing to the data loader. Sparse feature models (Naive Bayes, MLP, SVM, Logistic Regression, Decision Tree, Random Forest) convert this text to a fixed-size TF-IDF vector via the shared `fit_tfidf_features()` utility. Sequence models tokenize and index the text into fixed-length integer sequences.

Output: A single integer class prediction in the range $\{0, 1, 2, 3, 4\}$, corresponding to star ratings 1 through 5. Labels are zero-indexed internally throughout the codebase; the mapping to human-readable star ratings is applied only at evaluation and display time.

2.5 Class Labels

Label ID	Class Name	Meaning
0	1 star	Very negative review
1	2 stars	Negative review
2	3 stars	Mixed or neutral review
3	4 stars	Positive review
4	5 stars	Very positive review

2.6 Basic Dataset Statistics

The dataset is approximately balanced across all five classes. Each class accounts for exactly 20% of both the training and test partitions, with minor variation. Training class counts are approximately 130,000 reviews per class before the validation split is carved out. This near-uniform distribution means accuracy is a meaningful primary metric and no class weighting or oversampling is required.

Reviews vary substantially in length. Short reviews of one to two sentences are common in the extreme rating classes (1 star and 5 stars), where the reviewer's sentiment is unambiguous and concisely expressed. Middle-rating reviews (2, 3, and 4 stars) tend to be longer and more qualified, mixing positive and negative observations, which contributes to the difficulty of separating these classes.

2.7 Project Goals

The primary goal is to implement and compare ten machine learning models spanning traditional and deep learning approaches on a shared text classification benchmark. Each model is evaluated under a consistent experimental protocol using the same dataset splits, evaluation metrics, and result logging infrastructure. The secondary goal is to quantify the tradeoff between model complexity, input representation quality, and predictive performance, with particular attention to how much improvement sequence-aware deep learning models provide over bag-of-words baselines.

3. Implementation Description

3.1 Repository and Codebase Overview

3.1.1 Repository Structure

The codebase is organized such that each model is in its own directory and utilities that are shared across several models are found in `utils/`:

```
CAP5610TermProject/
|-- Linear_SVM/
|-- Kernel_SVM/
|-- Naive_Bayes/
|-- Decision_Tree/
|-- Random_Forest/
|-- Logistic_Regression/
|-- LSTM/
|-- MultiLayer_Perceptron/
|-- CNN/
|-- Transformers/
|-- Large_Language_Model/
|-- utils/
|-- requirements.txt
`-- README.md
```

3.1.2 Shared Utilities

The `utils/` package exports a common API that all models scripts can utilize:

```
utils/
|-- args.py                    # common CLI flags
|-- data.py                    # HuggingFace yelp_review_full loader + split
```

```
|-- embeddings.py              # GloVe / fastText loaders for CNN
|-- evaluation.py              # metrics, plots, results logging
|-- randomness.py              # DEFAULT_SEED, set_random
|-- text_features.py           # shared TF-IDF vectorizer
|-- timer.py                   # timed_step context manager
`-- tuning.py                  # Optuna driver + best-config persistence
```

3.1.3 Model Specific Directories

Each model's directory contains its script, a tuning log, a results log, a saved best-config, and a confusion matrix for the best final result. For example, `CNN/`:

```
CNN/
|-- CNN.py                     # runnable model script
|-- confusion_matrix_cnn.png   # confusion matrix from best final run
|-- best_config.json           # winning hyperparameters from tuning
|-- results_log.md             # every runs metrics, params, runtime
`-- tuning_log.md              # every Optuna trials params + score
```

3.2 Data Preparation and Experimental Pipeline

3.2.1 Data Loading Workflow

All models read the dataset through a single entry point: `utils.data.load_yelp_data()`, which calls `datasets.load_dataset("yelp_review_full")`. The dataset downloads on first use and is cached locally. Following runs then read from cache.

3.2.2 Training/Validation/Test Protocol

The official split of the dataset is 650,000 training reviews and 50,000 test reviews. The loader then takes an additional validation split (by default 10%) out of the training partition using `sklearn.model_selection.train_test_split`. The test partition is never touched during tuning. Validation runs use a default 150,000-review subsample of training data for fast iteration; final runs use the full 650,000-review training set (unless specified otherwise for some models) and evaluate on the held-out 50,000-review test partition (triggered by `--final`).

3.2.3 Text Preprocessing

Sparse-feature models (Logistic Regression, Naive Bayes, Decision Tree, Random Forest, Linear/Kernel SVM, MLP) consume TF–IDF features built directly from raw review text by `TfidfVectorizer`, which lowercases, tokenizes, and applies `strip_accents="unicode"` internally. Sequence models (CNN, LSTM, Transformer) lowercase the text and replace punctuation with whitespace before tokenizing on whitespace, then map tokens to fixed-size index sequences (CNN: `max_len = 256`; LSTM: `max_len = 512`; Transformer: `max_len = 160`). Tokens outside of the vocabulary collapse to `<unk>` and zero-padded positions are masked at the embedding layer. The decoder-only LLM (default `distilgpt2`) bypasses this pipeline entirely and instead applies the model's pretrained HuggingFace `AutoTokenizer` (subword BPE) directly to the raw text, padding/truncating to `max_length = 256`.

3.2.4 Feature Engineering

TF–IDF: `fit_tfidf_features()` fits a single `TfidfVectorizer` with `sublinear_tf=True` (dampens the contribution of repeated words within a review via $1 + \log(\text{tf})$) and four tunable parameters: `max_features` (cap on the vocabulary size), `ngram_range` (whether to include bigrams in addition to unigrams), `min_df` (drop terms appearing in fewer than this many documents, removing rare/typo tokens), and `max_df` (drop terms appearing in more than this fraction of documents, removing extremely common words). Final feature dimensionalities range from 10,000 (MLP) to 40,000 (Decision Tree, Random Forest, Logistic Regression).

Word vocabulary: Each word-level sequence model builds its own top N vocabulary from the training text with index 0 reserved for the padding token (`<pad>`) and index 1 reserved for the unknown word token (`<unk>`). CNN: 25,000 words (fixed); LSTM: 30,000 words (tuned over 10k–75k); Transformer: 15,000 words (tuned over 15k–30k). Reviews are truncated or zero-padded to the model's `max_len`.

Embeddings: The CNN supports either a randomly initialized 256-dimensional embedding lookup (trained from scratch) or one of several pretrained sources: GloVe 6B 100d/300d, GloVe 42B 300d, GloVe 2024 WikiGigaword 300d, and fastText wiki-news subword 300d, which can all be selected with mutually exclusive CLI flags. Pretrained matrices are loaded into the embedding layer at model construction time. The LSTM uses a trainable `nn.Embedding` lookup tuned over 128, 256, 300 dimensions with no pretrained option. The Transformer learns both token and positional embeddings from scratch (token embedding tuned over 128, 256 dimensions, positional embedding sized to `max_len`). The decoder-only LLM gets its subword embeddings from the pretrained `distilgpt2` checkpoint and tunes them jointly with the rest of the model.

3.2.5 Randomness and Reproducibility

A single global seed (`DEFAULT_SEED = 0`) is set at the start of every run via `set_random_seed()`, which seeds all of the following: Python's `random`, NumPy, and Torch (including CUDA), and pins `torch.backends.cudnn.deterministic = True` with `benchmark = False`. The same seed is passed into the dataset splitter and into every Optuna study, so re-running any script reproduces the same train/validation split and the same trial order.

3.2.6 Hyperparameter Tuning Workflow

Tuning is invoked with the `--tune` flag and uses Optuna Bayesian optimization through `utils.tuning.tune_model()`. Each model defines an `objective()` that samples a configuration, builds features, trains on a subset of the training data ranging from roughly 6,000 reviews (LLM) to 650,000 reviews (CNN, Linear SVM) depending on the model's compute cost per trial, and returns macro F1 on the validation split. Trial budgets range from 6 (Logistic Regression) to 30 (CNN), with most models in the 15–20 range. When tuning completes, `save_best_config()` writes the winning configuration plus metadata (seed, embedding source, train size, validation split, trial count, best macro F1) to `best_config.json`, and the tuning helper appends a Markdown summary of all trials to `tuning_log.md`. Following runs load that JSON automatically unless `--default` is passed. Naive Bayes is the exception: `--tune` runs a manual alpha sweep over a fixed grid rather than an Optuna study, and writes `nb_alpha_sweep.png` instead of a tuning log.

3.3 Traditional Machine Learning Models

3.3.1 Logistic Regression

Folder/Script: `Logistic_Regression/Logistic_Regression.py`
Team Member: Mohammed Mamdouh

Model Description:
A multinomial logistic regression classifier from `sklearn.linear_model.LogisticRegression`. The model learns one linear score per class over the TF–IDF feature vector and applies a SoftMax based cross entropy objective for review rating prediction. Unlike Naive Bayes, it does not assume feature independence, but it is still a linear bag of words model and therefore only captures limited local order through the chosen n-gram features.

Input Representation: Sparse TF–IDF features built directly in the script with `TfidfVectorizer`. The vectorizer uses `sublinear_tf=True`, `strip_accents="unicode"`, and `dtype=np.float32`. The tuned configuration keeps 40,000 features, includes unigrams and bigrams, and sets `min_df = 2`, `max_df = 0.9`.

Training Procedure: Validation runs load a 150,000 review subset and hold out 10% for validation where final runs fit on the full 650,000-review training split and evaluate on the 50,000 review test split. Optimization is handled by scikit-learn rather than a manual training loop. The script uses `lbfgs` for `l2` regularization and switches to `saga` when `elasticnet` is selected.

Hyperparameters: The main model hyperparameters are `C`, penalty type (`l2` or `elasticnet`), optional `l1_ratio` for elastic net mixing, and `max_iter`. TF–IDF preprocessing is tuned with the classifier through `tfidf_features`, `ngram_max`, `min_df`, and `max_df`. The convergence tolerance remains fixed at `1e-4`.

Tuning Strategy: Optuna Bayesian optimization with 6 trials on a 100,000 review training subset and a 10% validation split. Search space: `tfidf_features` $\in \{20000, 40000, 60000\}$, `ngram_max` $\in \{1, 2\}$, `min_df` $\in \{1, 2, 5\}$, `max_df` $\in \{0.9, 0.95, 1.0\}$, `C` $\in [0.25, 8.0]$ on a log scale, `penalty` $\in \{\texttt{l2}, \texttt{elasticnet}\}$, `max_iter` $\in \{200, 300, 500\}$, and `l1_ratio` $\in [0.05, 0.95]$ when elastic net is active. The saved best configuration uses `l2` regularization with $C \approx 0.379$ and achieved validation macro F1 of 0.6002 during tuning.

Implementation Notes: The full pipeline stays lightweight: TF–IDF fitting, model training, evaluation, and artifact saving all happen in one script while reusing the shared repo utilities for dataset loading, metrics, and logging. Because the representation remains sparse end-to-end and training is CPU-only, Logistic Regression serves as one of the fastest strong baselines in the project. Its final macro precision, macro recall, and macro F1 are all tightly clustered near 0.615, which suggests the model is already making fairly balanced decisions across classes and that most of the remaining limitations comes from the linear TF–IDF representation from being a non deep learning model rather than an easily fixable training issue.

3.3.2 Naive Bayes

Folder/Script: `Naive_Bayes/naive_bayes.py`
Team Member: Benedetto Falin

Model Description: Complement Naive Bayes (CNB) from `sklearn.naive_bayes.ComplementNB`. CNB is a variant of Multinomial Naive Bayes that estimates class-conditional probabilities using the *complement* of each class rather than the class itself. For each class c, it computes the likelihood of a feature given all other classes combined, which has been shown to produce better-calibrated weight estimates on imbalanced or heavily overlapping text corpora than standard Multinomial NB. The model assumes conditional independence of features given the class label (the Naive Bayes assumption), meaning word co-occurrence and order are not modeled.

Input Representation: TF-IDF vectors produced by the shared `fit_tfidf_features()` helper with `sublinear_tf=True`, `max_features = 20,000`, `ngram_range = (1, 2)`, `min_df = 5`, `max_df = 0.9`. Bigrams are included so that two-word phrases such as "not good" or "really great" are represented as single features, partially compensating for the model's inability to reason about word order at inference time. The resulting sparse matrix is passed directly to the sklearn fit and predict methods with no further transformation.

Training Procedure: CNB training is a single-pass frequency count followed by a log-probability computation – there is no iterative optimization. Final runs fit on the full 585,000-review training partition and evaluate on the 50,000-review test partition. Validation runs use the 150,000-review subsample with a 10% validation split for the alpha sweep. Training is CPU-only and completes in under 30 seconds on the full partition.

Hyperparameters: The only tunable model hyperparameter is `alpha`, the Laplace/Lidstone additive smoothing coefficient. Higher alpha values add more probability mass to unseen feature-class combinations, preventing zero-probability estimates that would dominate the log-likelihood. The four TF-IDF parameters listed above are fixed rather than tuned for this model.

Tuning Strategy: Rather than Optuna, a manual grid sweep over a fixed list of alpha values is performed when `--tune` is passed:

$$\alpha \in \{0.01,\ 0.05,\ 0.1,\ 0.3,\ 0.5,\ 1.0,\ 2.0,\ 5.0,\ 10.0,\ 15.0,\ 20.0,\ 25.0,\ 30.0,\ 35.0,\ 40.0,\ 45.0\ 50.0\}$$

For each candidate, the model is fit on the training subset and validation accuracy is recorded. The alpha achieving the highest validation accuracy is saved to `nb_best_config.json` and used for all subsequent runs. A validation accuracy versus alpha plot is saved as `nb_alpha_sweep.png`. The best alpha found was **45**. This value is higher than what was observed under the original unigram-only pipeline (where alpha of 30 was optimal), and the increase is attributable to the addition of bigrams in the TF-IDF feature set. Bigram features are considerably more sparse than unigram where a phrase like "not great" appears far less frequently than either constituent word, which reduces the average discriminative reliability of individual features and requires heavier smoothing to prevent the model from over-fitting to noisy frequency counts.

Implementation Notes: The unusually high optimal alpha is an interpretable finding in itself. Standard smoothing values (0.1 to 1.0) are sufficient when classes are well-separated in feature space. The fact that alpha must reach 45 before validation accuracy peaks indicates that individual word frequencies are weak discriminators across the five classes and heavy smoothing is needed to prevent the model from over-committing to any single term. This is consistent with the known difficulty of the middle star ratings (2, 3, and 4 stars), where reviews share substantial vocabulary regardless of the assigned rating.

3.3.3 Decision Tree

Folder/Script: `Decision_Tree/Decision_Tree.py`
Team Member: Keating Sane

Model Description: A single decision tree classifier from `sklearn.tree.DecisionTreeClassifier`. The tree is built top-down by repeatedly choosing one TF–IDF feature and a threshold that best separates the five star-rating classes (using Gini impurity), then recursing on each side. Predictions follow the chain of feature-vs-threshold tests from the root to a leaf, where the majority class is returned. Implemented as a baseline against the bagged ensemble (Random Forest).

Input Representation: TF–IDF vectors produced by the shared `fit_tfidf_features()` helper with `sublinear_tf=True`, `max_features = 40,000`, `ngram_range = (1, 2)`, `min_df = 10`, `max_df = 0.9`. No additional preprocessing.

Training Procedure: Final runs fit a single tree on the full 650,000-review training partition with the tuned config and evaluate on the 50,000-review test partition. Validation runs use a 150,000-review training subsample with a 10% validation split for fast iteration. CPU-only.

Hyperparameters: The tunable parameters are `max_depth` (maximum tree depth, which caps model capacity and the rate at which the tree can carve up the feature space), `min_samples_leaf` (minimum training rows per leaf, which prevents the tree from creating tiny leaves that memorize individual reviews), and the four TF–IDF parameters listed above. The splitting criterion is sklearn's default Gini impurity.

Tuning Strategy: Optuna Bayesian optimization with 20 trials on a 200,000-review tuning subset (10% validation split). Search space: `tfidf_features` $\in$ {5k, 10k, 20k, 40k}, `ngram_max` $\in$ {1, 2}, `min_df` $\in$ {3, 5, 10}, `max_df` $\in$ {0.85, 0.9, 0.95}, `max_depth` $\in$ {50, 100, 150, 300, None}, `min_samples_leaf` $\in [1, 10]$. macro F1 on the validation split was the objective.

Implementation Notes: Tree fitting on the 200k tuning subset takes well under a minute per trial on CPU, but the full 650k-review final fit grows to roughly an hour as the tree depth scales. With the best tuned settings, the model memorizes most of the training set: train accuracy is far above the macro F1 it achieves on validation/test. The model was kept as an interpretable baseline to compare with the ensemble.

3.3.4 Random Forest

Folder/Script: `Random_Forest/Random_Forest.py`
Team Member: Keating Sane

Model Description: Ensemble of decision trees from `sklearn.ensemble.RandomForestClassifier`. Each tree is fit on a bootstrap sample of the training rows with a random subset of features considered at every split (`max_features = "sqrt"`); class predictions are averaged across the forest. The randomized feature subsampling reduces the variance that the single Decision Tree exhibits.

Input Representation: TF–IDF vectors from `fit_tfidf_features()` with `sublinear_tf=True`, `max_features = 40,000`, `ngram_range = (1, 2)`, `min_df = 10`, `max_df = 0.85`. Identical pipeline to the single tree, with a slightly tighter `max_df`.

Training Procedure: Final runs fit a 500-tree forest on the full 650,000-review training partition with the tuned config and evaluate on the 50,000-review test partition. Validation runs use a 150,000-review training subsample with a 10% validation split. Tree fitting is parallelized across cores via `n_jobs = -1`; CPU-only.

Hyperparameters: The tunable parameters are mirrored from the Decision Tree, with two additions: `n_estimators` (number of trees in the forest) and `max_features` (number of features randomly considered at each split).

Tuning Strategy: Optuna Bayesian optimization with 20 trials on a 200,000-review tuning subset (10% validation split). Search space mirrors the Decision Tree's parameters and adds `n_estimators` $\in$ {100, 200, 300, 500} and `max_features` $\in$ {"sqrt", "log2"}. macro F1 on the validation split was the objective.

Implementation Notes: A 500-tree forest on the 200k tuning subset fits in roughly three minutes per trial with `n_jobs = -1`; the full 650k-review final fit grows to about 44 minutes. Bootstrap sampling plus the `max_features = "sqrt"` per-split feature subset force each tree to see a different slice of the data and consider a different subset of words at every split, so the trees make different mistakes and the variance that the single Decision Tree exhibits averages out.

3.3.5 Linear SVM

Folder/Script: `Linear_SVM/Linear_SVM.py`
Team Member:

Model Description:

Input Representation

Training Procedure

Hyperparameters

Tuning Strategy

Implementation Notes

3.3.6 Kernel SVM

Folder/Script: `Kernel_SVM/Kernel_SVM.py`
Team Member:

Model Description

Input Representation

Training Procedure

Hyperparameters

Tuning Strategy

Implementation Notes

3.4 Deep Learning Models

3.4.1 Multi-Layer Perceptron

Folder/Script: `MultiLayer_Perceptron/ml_perceptron.py`
Team Member: Benedetto Falin

Model Description: A fully connected feedforward neural network implemented in PyTorch (`torch.nn.Sequential`). The MLP takes a fixed-size TF-IDF vector as input and learns a non-linear mapping to five class logits via two hidden layers with batch normalization, ReLU activations, and dropout regularization. Unlike Naive Bayes, the MLP can learn interactions between input features, but like all TF-IDF-based models it has no access to word order or sequential context. Trained end-to-end with cross-entropy loss via backpropagation.

Input Representation: TF-IDF vectors from the shared `fit_tfidf_features()` helper with `sublinear_tf=True`, `max_features = 10,000`, `ngram_range = (1, 2)`, `min_df = 5`, `max_df = 0.9`. The sparse matrix is converted to a dense `float32` PyTorch tensor in chunks of 10,000 rows to avoid attempting a full dense allocation of approximately 20 GB. The resulting dense tensor is wrapped in a `TensorDataset` and served through a `DataLoader` with `pin_memory=True` for efficient GPU transfer.

Architecture:

$$\mathbb{R}^{10000} \xrightarrow{\text{Linear}} \mathbb{R}^{h_1} \xrightarrow{\text{BN, ReLU, Dropout}} \mathbb{R}^{h_2} \xrightarrow{\text{BN, ReLU, Dropout}} \mathbb{R}^{5}$$

where h_1 and h_2 are tunable hidden layer widths. Batch normalization is applied before each ReLU to stabilize training and reduce sensitivity to the learning rate. No softmax is applied at the output; `nn.CrossEntropyLoss` applies the log-softmax internally. The default architecture uses $h_1 = 1024$, $h_2 = 512$.

Training Procedure: Optimizer is Adam with a `ReduceLROnPlateau` scheduler that halves the learning rate when validation accuracy does not improve for 2 consecutive epochs. Early stopping with patience of 3 halts training when validation accuracy has not improved for 3 consecutive epochs, and the best-validation-accuracy checkpoint is restored before evaluation. Final runs train on the full 585,000-review training partition for up to 15 epochs and evaluate on the 50,000-review test partition. Validation runs use the 150,000-review subsample.

Hyperparameters: The tunable parameters and their Optuna search spaces are:

Parameter	Type	Search Space
`hidden1`	Categorical	{512, 1024, 2048}
`hidden2`	Categorical	{256, 512}
`dropout`	Float	$[0.1, 0.6]$ in steps of 0.1
`lr`	Float (log)	$[10^{-4},\ 5 \times 10^{-3}]$
`weight_decay`	Float (log)	$[10^{-5},\ 10^{-2}]$

Tuning Strategy: Optuna Bayesian optimization with 20 trials via `utils.tuning.tune_model()`. Each trial trains for up to 5 epochs with patience of 2 on the 150,000-review validation subset and returns the best validation accuracy achieved. The TPE sampler biases subsequent trials toward configurations that performed well in earlier trials. The best configuration found was $h_1 = 512$, $h_2 = 512$, `dropout` $= 0.6$, `lr` $= 0.0001$, `weight_decay` $= 5 \times 10^{-5}$, achieving a validation accuracy of 62.30% versus the default baseline of 60.28%.

Implementation Notes Early runs without early stopping exhibited severe overfitting: training accuracy climbed above 85% while validation accuracy plateaued near 61%, indicating the model's capacity far exceeds what the TF-IDF representation can supply in useful signal. Batch normalization and increased dropout (0.6 in the best config) were the two changes that most reduced the train/val gap. The performance ceiling near 61% is consistent with published results for bag-of-words MLP approaches on Yelp Full and is attributed primarily to the loss of word order at the input stage rather than insufficient model capacity.

3.4.2 Convolutional Neural Network

Folder/Script: `CNN/CNN.py`
Team Member: Keating Sane

Model Description: A Kim (2014) style TextCNN implemented in PyTorch. An embedding layer maps token indices to dense vectors, several 1D convolutional filters of different kernel widths run in parallel over the embedded sequence (acting as learned n-gram detectors), max-over-time pooling collapses each feature map to a single value per filter, the pooled outputs are concatenated, passed through dropout, and projected to five class logits via a fully connected layer. Trained end-to-end with cross-entropy loss.

Input Representation: Reviews are lowercased, punctuation is replaced with whitespace, and the text is tokenized on whitespace. The training corpus produces a 25,000-word vocabulary with index 0 reserved for the padding token (`<pad>`) and index 1 reserved for the unknown-word token (`<unk>`). Each review is mapped to a 256-token index sequence (truncated or zero-padded). The embedding source is chosen at the CLI: either a randomly initialized 256-dimensional lookup (default) or one of five pretrained options: GloVe 6B 100d, GloVe 6B 300d, GloVe 42B 300d, GloVe 2024 WikiGigaword 300d, or fastText wiki-news subword 300d. Pretrained matrices are loaded directly into the embedding layer at model construction.

Architecture: Embedding $(V \times E) \rightarrow$ parallel `Conv1d` filters at multiple kernel widths $\rightarrow$ ReLU $\rightarrow$ max-over-time pooling per filter $\rightarrow$ concatenate pooled outputs $\rightarrow$ dropout $\rightarrow$ fully connected layer producing 5-class logits. Padding positions are masked at the embedding via `padding_idx = 0`. Optimizer is Adam; gradients are clipped to a max L2 norm of 1.0.

Training Procedure: Final runs train on the full 650,000-review training partition and evaluate on the 50,000-review test partition. Validation runs use a 150,000-review training subsample with a 10%

validation split. Each epoch ends with a held-out validation pass that tracks the lowest-loss checkpoint; training stops early if validation loss has not improved for 3 consecutive epochs, and the lowest-loss checkpoint is restored before evaluation. Auto-selects MPS, CUDA, or CPU at startup.

Hyperparameters: The tunable parameters are `num_filters` (number of feature maps per kernel width), `filter_sizes` (the set of kernel widths run in parallel, wider filters capture longer n-grams), `dropout` (regularization on the concatenated feature vector before the fully connected layer), `lr` (Adam learning rate), `batch_size`, and `epochs` (effective via early stopping). The embedding source is selected at the CLI rather than tuned; the choice is recorded in `best_config.json`'s metadata so following runs can detect a mismatch.

Tuning Strategy: Optuna Bayesian optimization with 30 trials on the full 650,000-review training partition (10% validation split). Search space: `num_filters` $\in \{100, 200, 300\}$, `filter_sizes` $\in \{$"3,4,5", "2,3,4,5"$\}$, `dropout` $\in [0.2, 0.6]$, `lr` $\in [10^{-4}, 10^{-2}]$ (log scale), `batch_size` $\in \{64, 128\}$, `epochs` $\in [5, 15]$. macro F1 on the validation split was the objective. The best epoch identified by early stopping is written back into `best_config.json` so final runs reproduce that training duration.

Implementation Notes: Training is GPU-bound; on an RTX 5090 a final fit on 650k reviews completes in under a minute per epoch. Because the embedding source materially affects training, the saved choice is recorded in `best_config.json`'s metadata so following runs use the same embeddings the config was tuned with.

3.4.3 Transformer (Encoder Only)

Folder/Script: `Transformers/Transformers.py`
Team Member:

Model Description

Input Representation

Architecture

Training Procedure

Hyperparameters

Tuning Strategy

Implementation Notes

3.4.4 Large Language Model (Decoder Only)

Folder/Script: `Large_Language_Model/Large_Language_Model.py`
Team Member: Mohammed Mamdouh

Model Description A decoder only transformer finetuned for 5 class review classification through HuggingFace's `AutoModelForSequenceClassification`. Although the backbone model is a standard language model, this implementation uses it as a classifier by attaching a sequence level classification head and optimizing cross entropy loss on our dataset.

Pretrained Backbone: Our base model is `distilgpt2`. At load time, the script rewrites the configuration for `num_labels = 5` and populates label name mappings so predictions align with the repository's shared evaluation utilities. If the tokenizer does not define a padding token, the end of sequence token is reused for padding.

Tokenization and Input Format: Raw review text is passed directly to the pretrained HuggingFace tokenizer, which performs subword BPE tokenization. Inputs are truncated to a tunable `max_length` (128, 192, or 256 during tuning) and stored as `input_ids` plus `attention_mask`. Padding is applied dynamically per batch through `DataCollatorWithPadding`, which keeps memory use lower than padding the full dataset to a fixed length in advance.

Training Procedure: Non final runs use a 20,000 review subset with a 10% validation split. Final runs load the full 650,000-review training split, reserve 5% of it as an internal validation set for early stopping, then evaluate the best checkpoint on the 50,000-review test split. Optimization uses AdamW with a linear warmup schedule, gradient clipping at 1.0, mixed precision on CUDA, and early stopping off of the validation macro F1. After training, the best validation checkpoint is restored before the final evaluation pass.

Hyperparameters: The tunable training parameters are `max_length`, `epochs`, `batch_size`, `eval_batch_size`, `learning_rate`, `weight_decay`, `warmup_ratio`, and `grad_accum_steps`. Patience is fixed at 2 in regular runs and reduced to 1 during tuning so poor trials end quickly.

Tuning Strategy: Optuna optimization with 8 trials on a 6,000 review subset and a 10% validation split. The backbone itself is fixed during each study where tuning only searches the fine-tuning configuration. Search space: `max_length` $\in \{128, 192, 256\}$, `epochs` $\in \{1, 2\}$, `batch_size` $\in \{2, 4\}$, `eval_batch_size` $\in \{4, 8\}$, `learning_rate` $\in [10^{-5}, 8 \times 10^{-5}]$, `weight_decay` $\in [10^{-4}, 5 \times 10^{-2}]$, `warmup_ratio` $\in [0, 0.12]$, and `grad_accum_steps` $\in \{1, 2, 4\}$. The saved best configuration for `distilgpt2` uses `max_length = 256`, `epochs = 2`, `batch_size = 2`, `eval_batch_size = 8`, learning rate $\approx 1.28 \times 10^{-5}$, weight decay ≈ 0.0053, and warmup ratio ≈ 0.0172, reaching macro F1 of 0.5926 during tuning.

Implementation Notes: The implementation pretokenizes the dataset into a small custom `Dataset` object and uses `DataCollatorWithPadding` so each batch is padded only to its local maximum sequence length. On CUDA, training uses automatic mixed precision through `torch.autocast` and `GradScaler` to reduce memory pressure. The best model state is tracked in memory using validation macro F1, restored after training, and also written to `Large_Language_Model/saved_model` so the exact fine-tuned checkpoint can be reused later.

3.5 Cross-Model Engineering Choices

3.5.1 Common CLI Design

Every model script imports `common_parser()` from `utils.args`, which registers four shared flags: `--tune` (run hyperparameter search), `--final` (full training set, evaluate on the test partition), `--discard` (run without writing any artifacts), and `--default` (ignore any saved `best_config.json` and use hard-coded defaults). Model-specific flags (e.g. the CNN's embedding-source selectors, the SVMs' `--size`, the LLM's checkpoint and resume flags) are layered on top of this shared parser. `--tune` is mutually exclusive with `--final` and `--default`.

3.5.2 Logging and Artifact Saving

Run output is structured by helpers in `utils.evaluation`: `print_run_header()` prints the model name and contextual info; `print_value_section()` prints the parameter table; `timed_step()` prints elapsed time for each step during loading/training. After evaluation, `save_results()` appends a Markdown section to `results_log.md` with accuracy, macro precision, macro recall, macro F1, runtime, parameters, and metadata; the same helper guards against overwriting a better run by comparing macro F1 against the existing entry. For final runs, `plot_confusion_matrix()` writes a PNG figure into the model folder. `--discard` bypasses all writes.

3.5.3 Hardware/Runtime Considerations

Sparse feature models run on CPU and parallelize through `n_jobs=-1` where supported. The CNN, LSTM, Transformer, MLP, and LLM auto-select MPS, CUDA, or CPU at startup and use a fixed size token batch loader. Final run hardware varied: NVIDIA RTX 5090 (Linear SVM, CNN), AMD Ryzen 7 9800X3D (Decision Tree, Random Forest), AMD Ryzen 7 7800X3D (Naive Bayes, MLP), Tesla T4 (Transformer), and Intel i7-1165G7 (Kernel SVM). Final runtimes are recorded per-run in each `results_log.md`.

4. Usage Guide

This section explains how a user can setup our project to reproduce the results in this report.

4.1 Environment Setup

4.1.1 Prerequisites

Python 3.11 or newer, `pip`, and roughly 5 GB of free disk space for the cached dataset and pretrained embedding files. A CUDA or MPS capable GPU is recommended for the CNN, Transformer, and LLM but is not required. All other models run on CPU.

4.1.2 Installation

```
# clone
git clone <repo-url> && cd CAP5610TermProject

# create env (Python 3.11)
python -m venv .venv && source .venv/bin/activate

# install deps
pip install -r requirements.txt
```

4.1.3 Dataset Access

The dataset is fetched on first run via `datasets.load_dataset("yelp_review_full")` from HuggingFace. No credentials or manual downloads are required; following runs read from the local HuggingFace cache. Pretrained GloVe and fastText files used by the CNN download on demand the first time a corresponding embedding flag is passed.

4.2 Common Run Modes

Flag	Purpose	Notes
`--tune`	Run the Optuna hyperparameter search and save the best config.	Trains on a model-specific subset and evaluates on the validation split.
`--final`	Train on the full 650,000-review training partition (unless otherwise specified for some models) and evaluate on the 50,000 review test partition.	Loads the saved `best_config.json` unless `--default` is passed. Writes a confusion matrix PNG.
`--discard`	Skip writing any artifacts (results log, tuning log, best config, confusion matrix).	Useful for debugging.
`--default`	Ignore saved `best_config.json` and use the hard-coded defaults in the script.	Useful when rerunning with a different embedding source than the one the saved config was tuned with (e.g. CNN), or when comparing tuned vs. untuned baselines.

4.3 How to Run Each Model

Every model is invoked with `python <Directory>/<Script>.py` plus the desired flag combination. A typical workflow is to run `--tune` once to populate `best_config.json`, then run `--final` to produce the test-set numbers reported in Section 6.

Inputs: raw review text loaded automatically from the HuggingFace cache; for the CNN, an optional pretrained embedding flag.

Outputs: a printed run header, parameter table, timings, evaluation metrics, and (for final runs) a confusion matrix.

Artifacts produced: an appended section in the models `results_log.md`; for tuning runs, an appended section in `tuning_log.md` and a refreshed `best_config.json`; for final runs, `confusion_matrix_*.png`.

```
# Hyperparameter search (one time per model)
python Logistic_Regression/Logistic_Regression.py --tune
python Naive_Bayes/naive_bayes.py --tune
python Decision_Tree/Decision_Tree.py --tune
python Random_Forest/Random_Forest.py --tune
python Linear_SVM/Linear_SVM.py --tune
python Kernel_SVM/Kernel_SVM.py --tune
python MultiLayer_Perceptron/ml_perceptron.py --tune
python LSTM/LSTM.py --tune
python CNN/CNN.py --tune --glove-2024-wikigiga
```

```
python Transformers/Transformers.py --tune
python Large_Language_Model/Large_Language_Model.py --tune

# Final test-set evaluation using the saved best config
python Logistic_Regression/Logistic_Regression.py --final
python Naive_Bayes/naive_bayes.py --final
python Decision_Tree/Decision_Tree.py --final
python Random_Forest/Random_Forest.py --final
python Linear_SVM/Linear_SVM.py --final
python Kernel_SVM/Kernel_SVM.py --final
python MultiLayer_Perceptron/ml_perceptron.py --final
python LSTM/LSTM.py --final
python CNN/CNN.py --final --glove-2024-wikigiga
python Transformers/Transformers.py --final
python Large_Language_Model/Large_Language_Model.py --final

# Debugging: validation-mode run without writing anything
python CNN/CNN.py --discard --default
```

Additional flags. A few models layer extra options on top of the four shared flags:

- **CNN** (mutually exclusive embedding source selectors): `--glove-6b-100d`, `--glove-6b-300d`, `--glove-42b`, `--glove-2024-wikigiga`, `--fasttext-wiki-subword`. Omitting all of them uses a randomly initialized 256-dimensional embedding lookup.
- **Linear SVM** and **Kernel SVM**: `--size N` overrides the training subset size (defaults: 650,000 for Linear SVM, 10,000 for Kernel SVM).
- **Large Language Model**: `--model-name NAME` (default `distilgpt2`) chooses the decoder-only base model; `--smoke` runs a tiny model on a tiny subset for sanity checks; `--train-size N` overrides the non-final training subset size; `--trials N` sets the number of Optuna trials in `--tune` mode (default 8); `--resume` resumes from `Large_Language_Model/saved_model` and `--resume-from PATH` resumes from an arbitrary checkpoint. `--tune` additionally blocks `--resume`/`--resume-from`, and `--final` blocks `--smoke`.

4.4 Expected Output Files

Type	Location	Purpose
Best configuration	`<Model>/best_config.json`	Hyperparameters and metadata from the most recent tuning run.
Tuning logs	`<Model>/tuning_log.md`	Markdown record of every Optuna trial, noting parameters, val macro F1, and the runtime. Naive Bayes is the exception: it runs a manual alpha sweep producing `nb_alpha_sweep.png` instead.
Results logs	`<Model>/results_log.md`	Markdown record of every best validation and final run: accuracy, macro precision/recall/F1, parameters, metadata, runtime.
Confusion matrix	`<Model>/confusion_matrix_*.png`	PNG figure written on final runs.

5. Evaluation Metrics

5.1 Notation

5.2 Accuracy

Formula:

Interpretation:

5.3 Macro Precision

Formula:

Interpretation:

5.4 Macro Recall

Formula:

Interpretation:

5.5 Macro F1-Score

Formula:

Interpretation:

5.6 Confusion Matrix

Definition/Formula:

Interpretation:

5.7 Why These Metrics Fit This Project

6. Results

6.1 Validation Experiments

6.2 Final Experiments

6.3 Hardware and Software Environment

6.4 Tuning Budget

6.5 Experiment Tracking

6.6 Comparison Table

Item	Value
OS	
Python	
CPU	
GPU	
Key Libraries	

Model	Split	Accuracy	Macro Precision	Macro Recall	Macro F1	Runtime	Notes
Logistic Regression							
Naive Bayes							
Decision Tree							
Random Forest							
Linear SVM							
Kernel SVM							
Multi-Layer Perceptron							
CNN							
Transformer							
Large Language Model							

6.7 Per-Model Result Summaries

6.7.1 Logistic Regression

6.7.2 Naive Bayes

6.7.3 Decision Tree

6.7.4 Random Forest

6.7.5 Linear SVM

6.7.6 Kernel SVM

6.7.7 Multi-Layer Perceptron

6.7.8 Convolutional Neural Network

6.7.9 Transformer

6.7.10 Large Language Model

6.8 Visualizations

6.8.1 Confusion Matrices

6.8.2 Tuning Curves/Loss Curves/Accuracy Curves

6.9 Comparative Discussion

6.9.1 Best-Performing Models

6.9.2 Worst-Performing Models

6.9.3 Traditional ML vs. Deep Learning

6.9.4 Performance vs. Compute Tradeoffs

6.9.5 Error Patterns

7. Conclusions & Future Work

7.1 Key Findings

7.2 Lessons Learned

7.3 Limitations

7.4 Future Improvements

7.5 Final Takeaway